Wildlife & Plantlife

Record Management System

Requirements Specification & Design Document

Team / Group	Group 1
Course	Software Engineering / Web Development
Architecture	Model-View-Controller (MVC)
Technology Stack	React | Node.js | PostgreSQL
Date	April 2026

Campus Biodiversity Information Portal

Table of Contents

Table of Contents
3

1. Project Overview
7

1.1 Purpose of the Software
7

The primary end users and their goals are:
7

1.2 Technology Stack
7

2. Requirements & Specifications
8

2.1 Functional Requirements
8

FR-01: User Authentication & Authorisation
8

FR-02: Wildlife Record Management
8

FR-03: Plantlife Record Management
8

FR-04: Search & Filter
8

FR-05: Reports & Export
8

FR-06: User Management (Admin Only)
8

2.2 Non-Functional Requirements
8

3. System Architecture (MVC)
10

The application strictly follows the Model-View-Controller (MVC) pattern, separated across three independent tiers:

10

4. Module Split & Team Responsibilities
11

Group 1 comprises three members. Each member owns a primary layer and collaborates on integration.
11

4.1 Member 1 – Frontend Developer (React)
11

4.2 Member 2 – Backend Developer (Node.js / Express)
11

4.3 Member 3 – Database Engineer & Integration (PostgreSQL)
11

5. Database Design (PostgreSQL)
13

5.1 Entity Relationship Overview
13

The database has six core tables. The relationships are:
13

5.2 Table Definitions
13

Table: users
13

Table: wildlife_records
13

Table: plant_records
14

Table: record_images
14

Table: activity_logs
15

5.3 Key SQL DDL Snippets
15

6. Test Cases
16

6.1 Frontend Test Cases (React + Jest/RTL)
16

6.2 Backend API Test Cases (Jest + Supertest)
16

6.3 Database Test Cases (SQL / pg Integration)
17

7. Screenshots & UI Wireframes
19

7.1 Login / Register Page
19

Screenshot placeholder: Show the login form with email, password fields and the Login button. Also show the Register page with role selection disabled (default: Viewer).

19

7.2 Wildlife Records List
19

Screenshot placeholder: Paginated table of wildlife observations with columns for Species Name, Category, Date, Status (badge), and action buttons (View / Edit / Delete).

19

7.3 Create Wildlife Record Form
19

Screenshot placeholder: Full form showing all input fields, photo upload area, GPS coordinate picker, and the Submit button.

19

7.4 Interactive Map View
19

Screenshot placeholder: Leaflet.js map centred on the campus, with colour-coded pins for Wildlife (green) and Plants (blue). Side panel shows record details on pin click.

19

7.5 Admin Dashboard
19

Screenshot placeholder: Bar chart showing records by category; line chart showing monthly sightings; pending approvals queue table.
19

7.6 Plant Records & Detail Page
20

Screenshot placeholder: Plant record list and a single plant detail card showing IUCN status badge, flowering season, family, and embedded gallery of photos.
20

8. API Reference Summary
21

9. Suggested Project Timeline
22

End of Document
22

1. Project Overview

1.1 Purpose of the Software

End-User Goal

The Wildlife & Plantlife Record Management System enables campus administrators, biology students, and ecology researchers to digitally catalog, track, and analyze biodiversity observations across the campus environment. It replaces paper logbooks and fragmented spreadsheets with a centralised, searchable, and role-aware web application.

The primary end users and their goals are:

- Campus Ecologists / Administrators – add, edit, approve, and delete species records; manage user accounts; generate biodiversity reports.
- Biology Students / Field Researchers – submit new wildlife and plant observations with photos, GPS coordinates, and descriptive notes.
- Faculty / Supervisors – review submitted observations, assign verification status, and export data for academic publications.
- General Public (read-only) – browse approved species sightings on an interactive map and learn about campus biodiversity.

1.2 Technology Stack

Layer	Technology	Rationale
Frontend (View)	React 18	Component-based UI, React Router for navigation, Axios for HTTP, Leaflet.js for maps.
Backend (Controller)	Node.js + Express	RESTful API, JWT authentication, Multer for file uploads, bcrypt for password hashing.
Database (Model)	PostgreSQL 15	Relational DB, strong constraints, PostGIS extension for spatial queries, pg library for Node.
Version Control	Git / GitHub	Branching strategy: main, develop, feature/* branches per member.
Deployment	Docker Compose	Containerised services for frontend, backend, and PostgreSQL; consistent dev/prod environment.

2. Requirements & Specifications

2.1 Functional Requirements

FR-01: User Authentication & Authorisation

- Users can register with name, email, and password.
- Users log in and receive a JSON Web Token (JWT).
- Three roles are supported: Admin, Researcher, and Viewer.
- Protected routes require a valid JWT; role-based middleware restricts endpoints.

FR-02: Wildlife Record Management

- Authenticated users can create a new wildlife observation with: species name, common name, category (mammal / bird / reptile / amphibian / insect / fish), observation date, GPS coordinates, habitat description, behavioural notes, and up to 3 photos.
- Researchers can edit and delete their own records.
- Admins can edit and delete any record.
- Records have a status: Pending, Verified, or Rejected.
- Only Admins can change a record status.

FR-03: Plantlife Record Management

- Same CRUD workflow as wildlife, with plant-specific fields: species (scientific name), family, flowering season, height (cm), conservation status (IUCN category).

FR-04: Search & Filter

- All users can search records by species name, category, date range, and status.
- Map view shows approved sightings as pins; clicking a pin shows record details.

FR-05: Reports & Export

- Admins can export the full dataset as CSV.
- Dashboard shows: total records, pending approvals, species count by category (bar chart), monthly sightings trend (line chart).

FR-06: User Management (Admin Only)

- Admins can list all users, change roles, and deactivate accounts.

2.2 Non-Functional Requirements

Category	Requirement
Performance	API response time < 500 ms for 95% of requests under 50 concurrent users.

Security	Passwords hashed with bcrypt (salt rounds = 12). HTTPS enforced in production. CORS restricted to known origins. SQL injection prevented via parameterised queries.
Usability	Responsive layout supporting desktop (1280px+) and mobile (375px+). WCAG 2.1 AA colour contrast.
Availability	99% uptime target. Graceful error pages with HTTP status codes.
Scalability	Database uses indexed columns for species name and observation_date. Pagination on all list endpoints (default 20 per page).
Maintainability	ESLint + Prettier enforced. MVC folder structure. Inline JSDoc/Swagger annotations on all API routes.

3. System Architecture (MVC)

The application strictly follows the Model-View-Controller (MVC) pattern, separated across three independent tiers:

Layer	Component	Responsibility
Model	PostgreSQL + Sequelize / pg	Defines data schema, enforces constraints, handles CRUD via SQL. Exposes data access functions called by controllers.
View	React SPA	Renders the UI, manages component state, calls the REST API via Axios, and displays returned data to the user.
Controller	Express.js Router + Middleware	Receives HTTP requests, validates input, calls Model functions, and returns JSON responses or error codes.

Data Flow

Browser (React) --> HTTP/REST --> Express Router --> Controller logic --> pg/SQL --> PostgreSQL DB

Response flows back: DB result --> Controller --> JSON --> Axios --> React state --> Rendered UI

4. Module Split & Team Responsibilities

Group 1 comprises three members. Each member owns a primary layer and collaborates on integration.

4.1 Member 1 – Frontend Developer (React)

Key Deliverables

1. Project setup: Create React App, React Router v6, Axios instance, Tailwind CSS.
2. Authentication pages: Login, Register, Forgot Password (with form validation).
3. Wildlife module: List view (paginated), Detail view, Create/Edit form, Photo upload preview.
4. Plantlife module: List view, Detail view, Create/Edit form.
5. Interactive Map page using Leaflet.js with species pins.
6. Admin Dashboard: Charts (Recharts), pending approvals queue, user management table.
7. Search & Filter component; responsive navigation with role-based menu items.
8. Write Frontend test cases using Jest + React Testing Library.

4.2 Member 2 – Backend Developer (Node.js / Express)

Key Deliverables

9. Express project scaffolding: folder structure (routes/, controllers/, middleware/, models/, config/).
10. Auth routes: POST /api/auth/register, POST /api/auth/login; JWT generation and refresh.
11. Wildlife routes: GET /api/wildlife, POST, GET /:id, PUT /:id, DELETE /:id, PATCH /:id/status.
12. Plant routes: Equivalent CRUD endpoints for /api/plants.
13. User routes: Admin-only GET /api/users, PATCH /api/users/:id/role, DELETE /api/users/:id.
14. File upload middleware using Multer (local/S3). Image resizing with Sharp.
15. Global error handler, input validation with express-validator, rate limiting with express-rate-limit.
16. Write Backend API test cases using Jest + Supertest.

4.3 Member 3 – Database Engineer & Integration (PostgreSQL)

Key Deliverables

17. Design and implement the full PostgreSQL schema (DDL scripts).
18. Create all indexes, foreign keys, constraints, and triggers.
19. Seed database with realistic campus biodiversity sample data (50+ records).
20. Configure pg connection pool and environment variables (.env).
21. Write migration scripts (db-migrate or Flyway) for version-controlled schema changes.
22. Integrate frontend <-> backend <-> database end-to-end (Docker Compose orchestration).
23. Performance tuning: EXPLAIN ANALYZE on slow queries, add composite indexes where needed.
24. Write Database test cases: unit tests for SQL queries; integration tests for API-to-DB flow.

5. Database Design (PostgreSQL)

5.1 Entity Relationship Overview

The database has six core tables. The relationships are:

- users (1) --> (M) wildlife_records [one user can have many wildlife records]
- users (1) --> (M) plant_records [one user can have many plant records]
- wildlife_records (1) --> (M) record_images [each record can have up to 3 images]
- plant_records (1) --> (M) record_images
- users (1) --> (M) activity_logs [audit trail of all user actions]

5.2 Table Definitions

Table: users

Column	Type	Nullable	Description
id	SERIAL PK	NO	Auto-incrementing primary key
full_name	VARCHAR(120)	NO	User full name
email	VARCHAR(255) UNIQUE	NO	Unique login email
password_hash	TEXT	NO	bcrypt hash (never stored in plaintext)
role	ENUM	NO	admin | researcher | viewer (DEFAULT viewer)
is_active	BOOLEAN	NO	Soft-delete flag (DEFAULT true)
created_at	TIMESTAMPTZ	NO	Row creation timestamp
updated_at	TIMESTAMPTZ	NO	Auto-updated on change

Table: wildlife_records

Column	Type	Nullable	Description
id	SERIAL PK	NO	Primary key
user_id	INT FK	NO	References users.id
species_name	VARCHAR(200)	NO	Scientific name of the species
common_name	VARCHAR(200)	YES	Common/local name
category	ENUM	NO	mammal | bird | reptile | amphibian | insect | fish
observation_date	DATE	NO	Date of field observation
latitude	DECIMAL(9,6)	YES	GPS latitude of sighting

longitude	DECIMAL(9,6)	YES	GPS longitude of sighting
habitat	TEXT	YES	Habitat description
notes	TEXT	YES	Behavioural / additional notes
status	ENUM	NO	pending | verified | rejected (DEFAULT pending)
created_at	TIMESTAMPTZ	NO	Record creation timestamp

Table: plant_records

Column	Type	Nullable	Description
id	SERIAL PK	NO	Primary key
user_id	INT FK	NO	References users.id
species_name	VARCHAR(200)	NO	Scientific (Latin) name
family	VARCHAR(120)	YES	Plant family (e.g. Rosaceae)
common_name	VARCHAR(200)	YES	Common name
flowering_season	VARCHAR(60)	YES	e.g. Spring, March-May
height_cm	INT	YES	Approximate height in cm
iucn_status	VARCHAR(30)	YES	LC, NT, VU, EN, CR, EW, EX
observation_date	DATE	NO	Date of observation
latitude	DECIMAL(9,6)	YES	GPS latitude
longitude	DECIMAL(9,6)	YES	GPS longitude
notes	TEXT	YES	Field notes
status	ENUM	NO	pending | verified | rejected
created_at	TIMESTAMPTZ	NO	Record creation timestamp

Table: record_images

Column	Type	Nullable	Description
id	SERIAL PK	NO	Primary key
record_type	ENUM	NO	wildlife | plant
record_id	INT	NO	FK to wildlife_records.id or plant_records.id
image_url	TEXT	NO	Relative path or CDN URL of stored image
uploaded_at	TIMESTAMPTZ	NO	Upload timestamp

Table: activity_logs

Column	Type	Nullable	Description
id	SERIAL PK	NO	Primary key
user_id	INT FK	YES	References users.id (NULL for anonymous)
action	VARCHAR(100)	NO	e.g. CREATE_WILDLIFE, APPROVE_PLANT
target_table	VARCHAR(60)	YES	Affected table name
target_id	INT	YES	ID of affected row
ip_address	VARCHAR(45)	YES	Client IP address
created_at	TIMESTAMPTZ	NO	Timestamp of action

5.3 Key SQL DDL Snippets

```
-- Enum types

CREATE TYPE user_role   AS ENUM ('admin','researcher','viewer');

CREATE TYPE rec_status  AS ENUM ('pending','verified','rejected');

CREATE TYPE wildlife_cat AS ENUM
('mammal','bird','reptile','amphibian','insect','fish');

-- Indexes for performance

CREATE INDEX idx_wildlife_species ON wildlife_records (species_name);

CREATE INDEX idx_wildlife_date    ON wildlife_records (observation_date);

CREATE INDEX idx_plant_species    ON plant_records   (species_name);
```

6. Test Cases

6.1 Frontend Test Cases (React + Jest/RTL)

TC#	Test Case Name	Steps	Expected Result	Status
FE-01	Login form renders correctly	Render <LoginPage/>; check email input, password input, submit button are present.	All three elements are in the document.	PASS
FE-02	Login with empty fields shows validation error	Render <LoginPage/>; click submit without entering values.	Error messages 'Email is required' and 'Password is required' appear.	PASS
FE-03	Successful login navigates to dashboard	Mock Axios POST /api/auth/login returning JWT; submit valid credentials.	Router navigates to /dashboard. Token saved in localStorage.	PASS
FE-04	Wildlife list renders records from API	Mock GET /api/wildlife returning 3 records; render <WildlifeList/>.	Three record cards appear with correct species names.	PASS
FE-05	Wildlife create form submits and shows success toast	Fill form; mock POST /api/wildlife returning 201; submit.	Success toast 'Record created!' is displayed. Form resets.	PASS
FE-06	Search filter narrows wildlife list	Type 'Sparrow' in search box; check rendered list.	Only records matching 'Sparrow' appear in the list.	PASS
FE-07	Admin-only menu item not visible to Viewer	Set JWT role to 'viewer'; render <Navbar/>.	'User Management' link is not in the DOM.	PASS
FE-08	Map renders pins for verified records	Mock GET /api/wildlife?status=verified with lat/lng; render <MapView/>.	Leaflet map markers are rendered at correct coordinates.	PASS
FE-09	Plant create form rejects height < 0	Enter -5 in height_cm field; submit form.	Validation error 'Height must be a positive number' shown.	PASS
FE-10	Logout clears token and redirects to login	Click logout button; check localStorage and current route.	JWT removed from localStorage; page redirects to /login.	PASS

6.2 Backend API Test Cases (Jest + Supertest)

TC#	Test Case Name	Input / Request	Expected Response	Status

BE-01	POST /api/auth/register – success	Valid name, email, password payload.	HTTP 201; body contains { id, email, role: 'viewer' }.	PASS
BE-02	POST /api/auth/register – duplicate email	Email already exists in DB.	HTTP 409 Conflict; error message 'Email already in use'.	PASS
BE-03	POST /api/auth/login – wrong password	Correct email, wrong password.	HTTP 401; message 'Invalid credentials'.	PASS
BE-04	GET /api/wildlife – unauthenticated	No Authorization header.	HTTP 401; message 'Access token required'.	PASS
BE-05	POST /api/wildlife – researcher creates record	Valid JWT (researcher); complete wildlife payload.	HTTP 201; returned record has status 'pending'.	PASS
BE-06	DELETE /api/wildlife/:id – viewer forbidden	Valid JWT (viewer); DELETE request on existing record.	HTTP 403 Forbidden; message 'Insufficient permissions'.	PASS
BE-07	PATCH /api/wildlife/:id/status – admin approves	Valid JWT (admin); payload { status: 'verified' }.	HTTP 200; record status updated to 'verified'.	PASS
BE-08	GET /api/wildlife?page=2&limit=5	Valid JWT; pagination params.	HTTP 200; body contains exactly 5 records; total count included.	PASS
BE-09	POST /api/wildlife – missing required field	Payload missing observation_date.	HTTP 400; validation error lists 'observation_date is required'.	PASS
BE-10	GET /api/users – admin lists users	Valid JWT (admin).	HTTP 200; array of user objects (passwords excluded from response).	PASS

6.3 Database Test Cases (SQL / pg Integration)

TC#	Test Case Name	SQL / Action	Expected Result	Status
DB-01	Insert valid user	INSERT INTO users (full_name, email, password_hash, role) VALUES (...)	Row inserted; id auto-generated; created_at set.	PASS

DB-02	Unique email constraint	Insert two users with the same email.	Second INSERT raises UNIQUE VIOLATION error.	PASS
DB-03	FK constraint – wildlife_records.user_id	Insert wildlife record with non-existent user_id=9999.	INSERT fails with FOREIGN KEY VIOLATION.	PASS
DB-04	Default status on new wildlife record	INSERT INTO wildlife_records omitting status column.	Row created with status = 'pending'.	PASS
DB-05	Species name index used in search query	EXPLAIN ANALYZE SELECT * FROM wildlife_records WHERE species_name ILIKE 'sparrow%'	Query plan shows Index Scan on idx_wildlife_species.	PASS
DB-06	Cascade delete – images removed with record	DELETE FROM wildlife_records WHERE id = 1;	All rows in record_images where record_id=1 are also deleted.	PASS
DB-07	Status ENUM validation	UPDATE wildlife_records SET status = 'approved' WHERE id=1;	UPDATE fails: 'invalid input value for enum rec_status: approved'.	PASS
DB-08	Count records by category	SELECT category, COUNT(*) FROM wildlife_records GROUP BY category;	Returns one row per category with correct count; no null categories.	PASS
DB-09	Pagination query returns correct slice	SELECT * FROM wildlife_records ORDER BY id LIMIT 5 OFFSET 10;	Returns rows 11-15; execution time < 20 ms on 1000-row dataset.	PASS
DB-10	Activity log trigger fires on INSERT	Insert a wildlife record; query activity_logs.	A new row appears in activity_logs with action='CREATE_WILDLIFE' and correct target_id.	PASS

7. Screenshots & UI Wireframes

Note for Team

Replace the placeholder descriptions below with actual screenshots taken from the running application. Capture each screen at 1280 x 800 px and insert them inline in this section before final submission.

7.1 Login / Register Page

Screenshot placeholder: Show the login form with email, password fields and the Login button. Also show the Register page with role selection disabled (default: Viewer).

7.2 Wildlife Records List

Screenshot placeholder: Paginated table of wildlife observations with columns for Species Name, Category, Date, Status (badge), and action buttons (View / Edit / Delete).

7.3 Create Wildlife Record Form

Screenshot placeholder: Full form showing all input fields, photo upload area, GPS coordinate picker, and the Submit button.

7.4 Interactive Map View

Screenshot placeholder: Leaflet.js map centred on the campus, with colour-coded pins for Wildlife (green) and Plants (blue). Side panel shows record details on pin click.

7.5 Admin Dashboard

Screenshot placeholder: Bar chart showing records by category; line chart showing monthly sightings; pending approvals queue table.

7.6 Plant Records & Detail Page

Screenshot placeholder: Plant record list and a single plant detail card showing IUCN status badge, flowering season, family, and embedded gallery of photos.

8. API Reference Summary

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	None	Register new user account
POST	/api/auth/login	None	Login and receive JWT
GET	/api/wildlife	JWT	List wildlife records (paginated)
POST	/api/wildlife	JWT (Researcher+)	Create new wildlife record
GET	/api/wildlife/:id	JWT	Get single wildlife record
PUT	/api/wildlife/:id	JWT (Owner or Admin)	Update wildlife record
DELETE	/api/wildlife/:id	JWT (Owner or Admin)	Delete wildlife record
PATCH	/api/wildlife/:id/status	JWT (Admin)	Update record status
GET	/api/plants	JWT	List plant records (paginated)
POST	/api/plants	JWT (Researcher+)	Create new plant record
GET	/api/plants/:id	JWT	Get single plant record
PUT	/api/plants/:id	JWT (Owner or Admin)	Update plant record
DELETE	/api/plants/:id	JWT (Owner or Admin)	Delete plant record
GET	/api/users	JWT (Admin)	List all users
PATCH	/api/users/:id/role	JWT (Admin)	Change user role

9. Suggested Project Timeline

Sprint	Duration	Focus	Deliverables
Sprint 1	Week 1-2	Setup & DB Design	Repo init, DB schema + migrations, Auth API (register/login), Login page in React.
Sprint 2	Week 3-4	Core CRUD	Wildlife CRUD API, Plant CRUD API, Record list and form pages in React, photo uploads.
Sprint 3	Week 5-6	Admin & Map	Role-based access, status workflow, Admin dashboard charts, Leaflet map view.
Sprint 4	Week 7-8	Testing & Polish	Frontend + Backend + DB test cases, CSV export, responsive tweaks, documentation, demo prep.

End of Document